

CYNA DEV

Dossier d'Architecture Technique

Document de Conception Technique (DAT / DCT) — Bloc 3

Projet : CYNA DEV — Plateforme e-commerce SaaS de cybersécurité

Groupe : Groupe 4 — Charlotte Moerman · Nathan Noelijaona · Karl Negrevergne · Ylan Pernot

Formation : Bachelor BSI — Institut Limayrac

Version : 1.0

1. Introduction

Ce document constitue le Dossier d'Architecture Technique (DAT) du projet CYNA DEV. Il complète la note de cadrage (Bloc 1) en détaillant les choix d'implémentation réellement retenus : architecture logicielle, modèle de données, API, sécurité, installation et exploitation.

Il s'adresse au jury du Bloc 3 ainsi qu'à toute personne souhaitant comprendre, installer ou faire évoluer la plateforme.

1.1 Rappel du contexte

CYNA est une entreprise de cybersécurité fondée en 2021, proposant des solutions SaaS de protection informatique (EDR, XDR, SOC) jusqu'ici commercialisées via un réseau commercial traditionnel. CYNA DEV a pour objectif de digitaliser cette offre via une plateforme e-commerce sécurisée, permettant aux clients de découvrir, souscrire et gérer leurs abonnements en autonomie.

1.2 Périmètre couvert par ce document

- Architecture applicative et modèle de données
- Choix technologiques et justifications
- Documentation des endpoints de l'API REST, dont le chatbot IA
- Guide d'installation et de déploiement
- Supervision et observabilité (métriques, logs centralisés)
- Plan de sécurité
- Plan de maintenance et d'évolutivité

2. Architecture du système

2.1 Vue d'ensemble — architecture en 3 couches

L'application suit une architecture classique en trois couches, conteneurisée via Docker Compose :

Couche	Rôle	Technologie
Présentation	Interface utilisateur, responsive et mobile-first	React 18 + Vite, servi en production par Nginx
Applicative	API RESTful : authentification, logique métier, paiement, validation	Node.js + Express 5
Données	Persistence des données personnelles, du catalogue et des transactions	PostgreSQL 16

Le back-end n'est jamais exposé directement sur Internet : seul le service frontend (Nginx) publie un port, et redirige les requêtes /api vers le service backend au sein du réseau Docker interne (app_network).

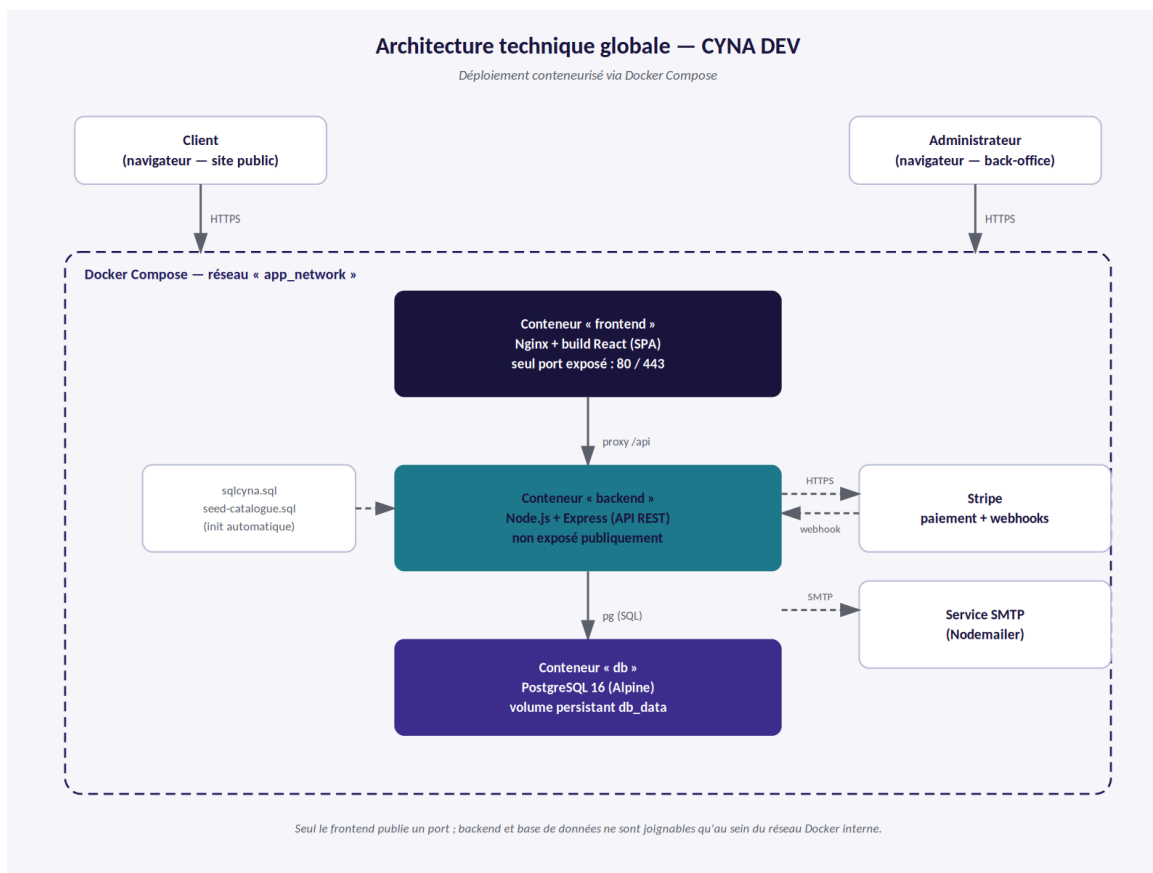


Figure 1 — Architecture technique globale et déploiement Docker Compose

2.1 bis — Diagramme de cas d'utilisation

Le diagramme ci-dessous synthétise les cas d'usage couverts par la plateforme, pour les deux profils d'acteurs : le client (visiteur ou compte connecté) et l'administrateur, qui hérite de l'ensemble des cas d'usage du client en plus de ses propres fonctions de back-office.

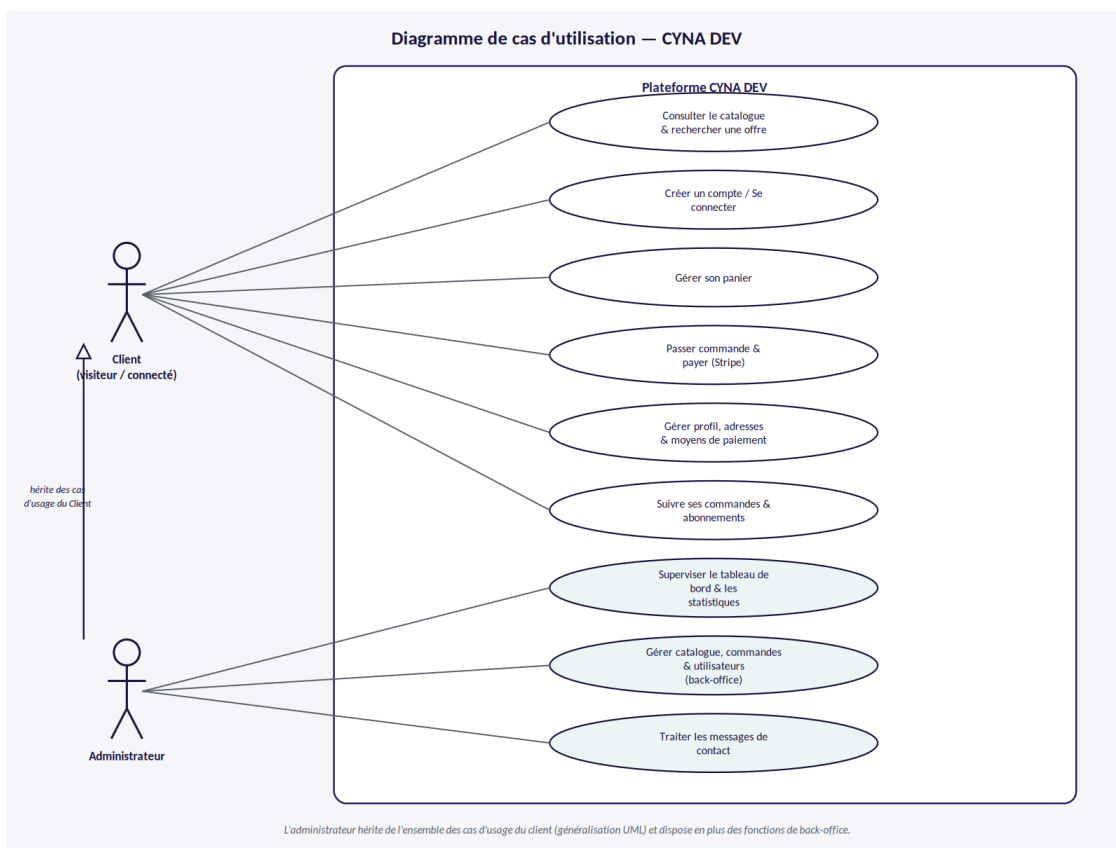


Figure 2 — Diagramme de cas d'utilisation UML

2.2 Architecture applicative détaillée

Back-end — organisation en couches (MVC)

Couche	Répertoire	Responsabilité
Routes	src/routes/	Déclarent les endpoints HTTP et branchent middlewares de validation/authentification
Controllers	src/controllers/	Reçoivent la requête HTTP, orchestrent l'appel aux services, formatent la réponse
Services	src/services/	Contiennent la logique métier pure et l'accès aux données
Middlewares	src/middlewares/	auth.js (vérification JWT), validate.js (retour des erreurs de validation), errorHandler.js (gestion centralisée des exceptions)
Config	src/config/	db.js (pool PostgreSQL), env.js (variables d'environnement), stripe.js (client Stripe)

Front-end — organisation des sources

Répertoire	Contenu
src/pages/	14 pages publiques (Home, Catalogue, Produit, Recherche, Panier, Checkout, Confirmation, Login, Register, ForgotPassword, ResetPassword, ConfirmEmail, Contact, PageStatique)
src/pages/account/	6 pages de l'espace client (MonCompte, Profil, Adresses, Abonnements, Paiements, Commandes)

Répertoire	Contenu
src/pages/admin/	10 pages du back-office (Dashboard, Produits, ProduitForm, Categories, Carrousel, Accueil, Commandes, Abonnements, Utilisateurs, Messages)
src/components/	Composants réutilisables : ui/ (Button, Input, Modal, Spinner, ProductCard...), layout/ (Header, Footer, Layout), checkout/ (PaiementStripe), admin/ (AdminLayout, AdminSidebar, Charts)
src/context/	AuthContext.jsx (session utilisateur), CartContext.jsx (panier) — état global géré via React Context API, sans dépendance externe
src/api/	axios.js (client HTTP configuré), index.js (appels API centralisés)
src/routes/AppRouter.jsx	Déclaration des routes et de leur protection (publiques / privées / admin)

2.3 Flux de données — cas d'usage clé : souscription à une offre

Le schéma logique ci-dessous décrit le flux de données lors d'une souscription, de bout en bout :

Étape	Composant	Action
1	Front (React)	L'utilisateur ajoute une offre au panier (CartContext) puis valide le checkout
2	Front → API	POST /api/commandes avec l'adresse, le moyen de paiement et les lignes de commande
3	Back (commandes.controller)	Validation des données (express-validator), création de la commande en base
4	Front (Stripe.js)	Saisie sécurisée de la carte, confirmation du PaymentIntent auprès de Stripe
5	Stripe → Back	Notification asynchrone via POST /api/paiement/webhook (signature vérifiée)
6	Back (webhook.service)	Mise à jour du statut de la commande / activation de l'abonnement
7	Back → Front	Le client consulte la confirmation et retrouve la commande dans son historique

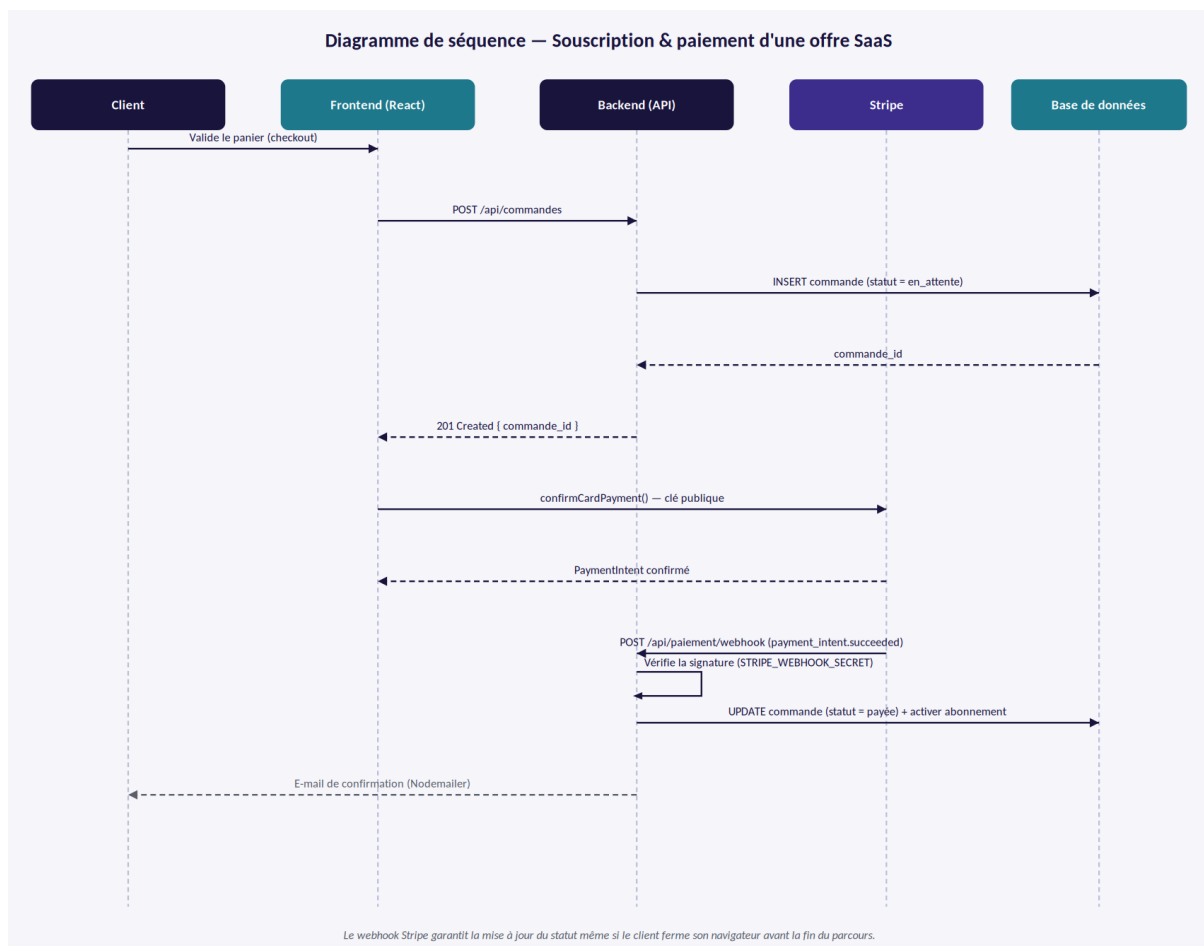


Figure 3 — Diagramme de séquence : souscription et paiement d'une offre

2.4 Communication avec les services externes

- Stripe — création de PaymentIntent, gestion des abonnements, réception des webhooks (paiement réussi/échoué, abonnement mis à jour/résilié)
- SMTP (Nodemailer) — e-mail de confirmation d'inscription, réinitialisation de mot de passe, notifications de commande

3. Modèle de données

Le schéma relationnel a été modélisé sous PGERD (bddcyna.pgerd) puis implémenté en PostgreSQL (sqlcyna.sql). Il comprend 16 tables, réparties en 5 grands domaines :

Domaine	Tables	Description
Comptes & sécurité	Utilisateurs, Tokens_Email, Sessions	Comptes clients/admin, tokens de confirmation d'e-mail et de réinitialisation, sessions actives (refresh tokens)
Adresses & paiement	Adresses, Methodes_Paiement	Adresses de facturation, moyens de paiement enregistrés (référence Stripe uniquement, jamais de PAN complet)
Catalogue	Categories, Produits, Produits_Images, Prix	Offres SaaS (SOC/EDR/XDR), catégorisation, visuels, grille tarifaire (mensuel/annuel)
Commandes & abonnements	Commandes, Commandes_Lignes, Abonnements, Factures	Cycle de vie complet d'une commande et des abonnements associés, facturation
Contenu & support	Carrousel, Contenu_Accueil, Messages_Contact	Contenu éditorial de la page d'accueil piloté depuis le back-office, messages du formulaire de contact

Le détail des cardinalités, clés primaires/étrangères et contraintes est disponible dans le fichier bddcyna.pgerd (modèle PGERD) et dans sqlcyna.sql (script de création exécuté automatiquement par Docker au démarrage du conteneur PostgreSQL).

4. Choix technologiques

Composant	Technologie	Justification
Front-end	React 18 + Vite	Écosystème mature, SPA performante, build de développement quasi instantané avec Vite
Routing	React Router 6	Standard de facto pour la navigation déclarative en React
État global	Context API	Suffisant pour le périmètre du projet (auth + panier) ; évite la complexité de Redux
Paiement (front)	@stripe/react-stripe-js	Formulaire de carte hébergé par Stripe (conformité PCI-DSS) sans manipulation de données bancaires côté CYNA
Back-end	Node.js + Express 5	Écosystème JavaScript unifié front/back, large communauté, montée en charge horizontale facilitée
Base de données	PostgreSQL 16	SGBD relationnel robuste, transactions ACID indispensables pour la gestion des commandes/paiements
Authentification	JWT (access + refresh) + bcrypt	Authentification stateless scalable ; bcrypt pour le hachage irréversible des mots de passe

Composant	Technologie	Justification
Paielement (back)	Stripe Node SDK	Délègue la conformité PCI-DSS à un tiers certifié, webhooks fiables
Sécurité HTTP	Helmet, express-rate-limit, express-validator	Durcissement des en-têtes HTTP, protection anti brute-force, validation systématique des entrées
Conteneurisation	Docker + Docker Compose	Environnements reproductibles (dev/staging/prod), isolation des services, déploiement simplifié
Reverse proxy	Nginx	Sert le build React statique et redirige les appels /api vers le backend

5. Documentation des API

L'API suit une convention REST sous le préfixe `/api`. Toutes les routes protégées exigent un en-tête d'authentification (JWT) vérifié par le middleware `verifyToken`. Les tableaux ci-dessous listent les endpoints réellement implémentés.

5.1 Authentification — `/api/auth`

Méthode	Route	Description	Protection
POST	<code>/register</code>	Inscription (mot de passe fort exigé)	5 req/h/IP
GET	<code>/confirm-email</code>	Confirmation du compte via token reçu par e-mail	Publique
POST	<code>/login</code>	Connexion (email + mot de passe)	10 req/15min/IP
POST	<code>/refresh</code>	Renouvellement du token d'accès via le refresh token	Publique
POST	<code>/logout</code>	Déconnexion, invalidation de la session	Publique
POST	<code>/forgot-password</code>	Envoi d'un lien de réinitialisation par e-mail	Publique
POST	<code>/reset-password</code>	Réinitialisation du mot de passe via token	Publique

5.2 Catalogue — `/api`

Méthode	Route	Description
GET	<code>/accueil</code>	Contenu dynamique de la page d'accueil (carrousel, sections)
GET	<code>/categories</code>	Liste des catégories de produits
GET	<code>/categories/:id/produits</code>	Produits d'une catégorie
GET	<code>/produits/recherche</code>	Recherche avancée dans le catalogue
GET	<code>/produits/:id</code>	Détail d'une offre SaaS

5.3 Utilisateurs — `/api/utilisateurs (authentié)`

Méthode	Route	Description
GET	<code>/me</code>	Profil de l'utilisateur connecté
PUT	<code>/me</code>	Modification du profil
PUT	<code>/me/mot-de-passe</code>	Changement de mot de passe (mot de passe actuel requis)
DELETE	<code>/me</code>	Suppression du compte
GET / POST	<code>/me/adresses</code>	Carnet d'adresses — liste / ajout
PUT / DELETE	<code>/me/adresses/:id</code>	Modification / suppression d'une adresse

Méthode	Route	Description
PUT	/me/adresses/:id/default	Définir l'adresse par défaut
GET / POST	/me/paiements	Moyens de paiement enregistrés — liste / ajout
DELETE	/me/paiements/:id	Suppression d'un moyen de paiement
PUT	/me/paiements/:id/default	Définir le moyen de paiement par défaut

5.4 Commandes & abonnements (authentifé)

Méthode	Route	Description
POST	/api/commandes	Création d'une commande (adresse, moyen de paiement, lignes de panier)
GET	/api/commandes	Historique des commandes de l'utilisateur
GET	/api/commandes/:id	Détail d'une commande
GET	/api/abonnements	Abonnements de l'utilisateur
GET	/api/abonnements/:id	Détail d'un abonnement
POST	/api/abonnements/:id/resilier	Résiliation d'un abonnement

5.5 Paiement — /api/paiement

Méthode	Route	Description
POST	/webhook	Réception des événements Stripe (signature vérifiée, corps brut) — paiement réussi/échoué, abonnement mis à jour/résilié

5.6 Contact — /api/contact

Méthode	Route	Description
POST	/	Envoi du formulaire de contact
POST	/chatbot	Message envoyé via le chatbot du site

5.7 Chatbot IA — /api/chatbot

Méthode	Route	Description
POST	/chat	Envoie un message utilisateur à l'assistant et renvoie sa réponse

Le chatbot s'appuie sur un modèle de langage auto-hébergé (Ollama, modèle llama3.2:1b) plutôt que sur une API tierce payante. Le contrôleur chatbot.controller.js construit dynamiquement un prompt injectant : l'identité du client s'il est connecté (prénom, e-mail extraits du JWT), et un extrait du catalogue actif (nom, catégorie, description, prix) pour que les réponses restent ancrées dans l'offre réelle de CYNA. Le prompt système restreint explicitement l'assistant aux sujets liés au site (produits, abonnements, commandes, sécurité, facturation, support) et lui interdit d'inventer des informations.

5.8 Endpoints techniques

Méthode	Route	Description
GET	/health	Vérification de disponibilité du service (utilisée par des sondes de supervision)
GET	/metrics	Métriques au format Prometheus (voir section Supervision et observabilité)

5.9 Back-office — /api/admin (authentifié, rôle administrateur)

Domaine	Routes principales	Opérations
Tableau de bord	GET /dashboard/stats	Statistiques globales
Produits	/produits, /produits/:id, /produits/:id/images	CRUD complet + gestion des images et de leur ordre
Prix	/prix/:id	Modification / suppression d'un tarif
Catégories	/categories, /categories/:id	CRUD + réorganisation de l'ordre d'affichage
Carrousel	/carrousel, /carrousel/:id	CRUD + réorganisation
Accueil	/accueil, /accueil/:cle	Édition du contenu dynamique de la page d'accueil
Utilisateurs	/utilisateurs, /utilisateurs/:id	Liste, détail, modification, anonymisation (RGPD)
Commandes	/commandes, /commandes/:id	Supervision des commandes clients
Abonnements	/abonnements, /abonnements/:id	Supervision des abonnements
Messages	/messages, /messages/:id	Consultation et traitement des messages de contact

6. Guide d'installation

6.1 Prérequis

- Docker & Docker Compose (méthode recommandée), ou Node.js 18+ et PostgreSQL 16 en local
- Un compte Stripe (mode test) pour les clés API
- Un accès SMTP (ou un service comme Mailtrap en développement) pour l'envoi d'e-mails

6.2 Variables d'environnement

Le backend valide au démarrage la présence de `DATABASE_URL`, `JWT_SECRET` et `JWT_REFRESH_SECRET` (voir `src/config/env.js`). Variables complètes (`backend/.env.example`) :

Variable	Rôle
PORT	Port d'écoute de l'API (3000 par défaut)
DATABASE_URL	Chaîne de connexion PostgreSQL
JWT_SECRET / JWT_EXPIRES_IN	Secret et durée de vie du token d'accès (15 min par défaut)
JWT_REFRESH_SECRET / JWT_REFRESH_EXPIRES_IN	Secret et durée de vie du refresh token (7 jours par défaut)
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET	Clés Stripe (secrète + signature webhook)
SMTP_HOST / PORT / USER / PASS	Configuration du serveur d'envoi d'e-mails
FRONTEND_URL	URL du frontend (utilisée pour CORS et les liens dans les e-mails)

Côté frontend (`frontend/.env.example`) : `VITE_API_URL` (URL de l'API), `VITE_STRIPE_PUBLIC_KEY` (clé publique Stripe), `VITE MOCK` (bascule vers des données de démonstration sans backend).

6.3 Lancement en local — via Docker (recommandé)

1. Cloner le dépôt et se placer dans le dossier `code/`
2. Copier `backend/.env.example` vers `backend/.env` et renseigner les valeurs
3. Copier `frontend/.env.example` vers `frontend/.env`
4. Lancer : `docker-compose up --build`
5. Le frontend est accessible sur `http://localhost`, l'API sur `http://localhost/api` (proxy Nginx)

Docker Compose orchestre 3 services : `db` (PostgreSQL 16 Alpine, initialisé automatiquement avec `sqlcyna.sql` et `seed-catalogue.sql`), `backend` (Express, non exposé publiquement) et `frontend` (build React servi par Nginx, seul service exposé sur le port 80).

6.4 Lancement en local — sans Docker

- Backend : `cd backend && npm install && npm run dev` (nodemon, rechargement automatique)
- Frontend : `cd frontend && npm install && npm run dev` (Vite, sur `http://localhost:5173`)

6.5 Déploiement en production

- Build du frontend : npm run build (généré dans frontend/dist, servi par Nginx via le Dockerfile dédié)
- Le backend tourne en mode conteneurisé, sans port exposé publiquement
- Mise en place d'un certificat HTTPS (Let's Encrypt ou équivalent) au niveau du reverse proxy
- Configuration du webhook Stripe pointant vers /api/paiement/webhook en production

7. Supervision et observabilité

Contrairement à une simple journalisation console, CYNA DEV intègre une chaîne d'observabilité complète, déployée via une stack Docker Compose dédiée (code/monitoring/).

7.1 Instrumentation applicative

Composant	Rôle
winston (backend/src/monitoring/logger.js)	Logs structurés au format JSON (timestamp, niveau, message, request_id, route) — chaque requête HTTP reçoit un identifiant unique (x-request-id) propagé dans les logs pour faciliter le traçage
prom-client (backend/src/monitoring/metrics.js)	Expose des métriques Prometheus sur GET /metrics : durée des requêtes HTTP (histogramme par méthode/route/code), nombre de requêtes en cours, mais aussi des métriques métier — commandes créées, montant total des ventes, erreurs de paiement par type, taille moyenne du panier, latence du service de paiement, sessions actives

7.2 Stack d'observabilité (code/monitoring/docker-compose.yml)

Outil	Rôle
Prometheus	Collecte et stocke les métriques exposées par le backend (scraping périodique de /metrics)
Grafana	Tableaux de bord de visualisation des métriques et des logs
Loki	Agrégation centralisée des logs applicatifs
Promtail	Collecte les logs des conteneurs Docker et les transmet à Loki
node-exporter	Métriques systèmes (CPU, mémoire, disque) de la machine hôte

Cette stack de supervision dépasse ce qu'exigeait explicitement le cahier des charges (qui ne demandait un outil de supervision précis que pour la filière Systèmes/Réseaux/Cloud) — elle constitue une valeur ajoutée du projet, cohérente avec les exigences de démonstration de la supervision lors de la soutenance du Bloc 3.

7.3 Chatbot IA auto-hébergé (Ollama)

Le service de chatbot (voir section 5.7) s'appuie sur un conteneur Ollama exécutant un modèle de langage en local (llama3.2:1b), plutôt que sur une API IA tierce payante. Un service ollama-init dédié télécharge automatiquement

le modèle au premier démarrage de la stack Docker. Ce choix élimine toute dépendance à un fournisseur externe et tout coût récurrent par appel, au prix d'une empreinte mémoire locale plus importante.

8. Plan de sécurité

8.1 Authentification & gestion de session

- Mots de passe hachés avec bcrypt — jamais stockés en clair
- Politique de mot de passe fort imposée à l'inscription : 8 caractères minimum, majuscule, minuscule, chiffre, caractère spécial (express-validator / isStrongPassword)
- JWT à courte durée de vie (15 min) + refresh token (7 jours) pour limiter la fenêtre d'exposition en cas de vol de token
- Limitation du débit (express-rate-limit) : 5 inscriptions/heure/IP, 10 connexions/15 min/IP — protection contre le brute-force
- Confirmation d'adresse e-mail obligatoire avant activation complète du compte

8.2 Protection des données en transit et au repos

- HTTPS/TLS en production (reverse proxy Nginx)
- Aucune donnée de carte bancaire ne transite ni n'est stockée par les serveurs CYNA : la saisie est déléguée à Stripe.js (conformité PCI-DSS)
- Vérification cryptographique de la signature des webhooks Stripe avant tout traitement (stripe.webhooks.constructEvent)
- Secrets (JWT, Stripe, SMTP) exclusivement stockés dans des variables d'environnement, jamais committés (voir .gitignore et .env.example)

8.3 Durcissement applicatif

Mesure	Outil	Protection apportée
En-têtes HTTP sécurisés	Helmet	CSP, X-Frame-Options, HSTS, désactivation du fingerprinting du framework
Validation systématique des entrées	express-validator	Prévention des injections et des données malformées sur toutes les routes sensibles
Requêtes paramétrées	pg (driver PostgreSQL)	Élimination du risque d'injection SQL
Cookies sécurisés	cookie-parser + httpOnly	Protection contre le vol de session via XSS
CORS restreint	cors (origin = FRONTEND_URL)	Empêche les requêtes cross-origin non autorisées
Limite globale de débit	express-rate-limit — 100 req/15 min/IP sur toute l'API	Protection générale contre les abus, en complément des limites spécifiques sur /auth/login (10/15min) et /auth/register (5/heure)
IP réelle derrière le proxy	app.set('trust proxy', 1)	Garantit que le rate-limiting utilise la véritable IP du client (transmise via X-Forwarded-For par le reverse proxy Nginx), et non l'IP interne du conteneur frontend

Mesure	Outil	Protection apportée
Gestion centralisée des erreurs	errorHandler.js	Évite la fuite d'informations techniques (stack traces) au client

8.4 Conformité RGPD

- Minimisation des données : seules les informations nécessaires à la gestion du compte et des commandes sont collectées
- Droit à l'oubli : endpoint DELETE /api/utilisateurs/me pour la suppression de compte ; anonymisation possible côté admin
- Droit d'accès et de rectification : endpoints GET/PUT /api/utilisateurs/me
- Politique de confidentialité à rédiger et rendre accessible sur le site (page statique)

9. Plan de maintenance et d'évolutivité

9.1 Maintenance corrective

- Suivi des anomalies via GitHub Issues, priorisation selon la criticité
- Logs applicatifs centralisés (backend/boot.log, err.log, out.log) pour le diagnostic
- Mises à jour régulières des dépendances (npm audit) pour corriger les vulnérabilités connues

9.2 Scalabilité

- Architecture stateless côté API (JWT) : possibilité de répliquer horizontalement le service backend derrière un load balancer
- Séparation stricte des services (db / backend / frontend) permettant de faire évoluer chacun indépendamment
- Base de données PostgreSQL : mise en place envisageable de réplicas en lecture si la charge l'exige

9.3 Évolutions envisagées

- Application mobile native (iOS / Android)
- Authentification à deux facteurs (2FA) généralisée
- Mise en place d'un pipeline CI/CD complet (tests automatisés + déploiement continu)
- Supervision applicative avancée (Grafana/Prometheus ou équivalent)
- Programme de fidélité et offres personnalisées

10. Annexes

10.1 Dépendances principales — backend (package.json)

bcrypt, cookie-parser, cors, dotenv, express, express-rate-limit, express-validator, helmet, jsonwebtoken, nodemailer, pdfkit, pg, prom-client, stripe, winston — devDependencies : nodemon.

10.2 Dépendances principales — frontend (package.json)

@stripe/react-stripe-js, @stripe/stripe-js, axios, react, react-dom, react-router-dom — devDependencies : vite, @vitejs/plugin-react.

10.3 Documents complémentaires

- Note de cadrage (Bloc 1) — Documentations.pdf
- Modèle de données PGERD — bddcyna.pgerd
- Script de création de la base — sqlcyna.sql
- Maquettes du site — Maquette_du_site.pdf
- Document de test, Document de suivi de projet, Document de fin de projet (documents séparés)